
Mental Accounts for Actions: EWA-Inspired Attention in Decision Transformers

Abstract

Transformers for control propagate credit through content-based attention yet lack an explicit memory of action-specific outcomes. We introduce Experience-Weighted Attraction with Vector Quantization for Online Decision Transformers (EWA-VQ-ODT). The method maintains a per-action mental account of recent success and failure. Continuous actions are routed to a small vector-quantized codebook by direct grid lookup, and each code holds an attraction scalar updated online by a simple recency-weighted rule with global decay each step and additive reinforcement when the code is chosen. We add these attractions as a small bias to the attention columns for action tokens before the softmax, leaving the backbone and learning objective unchanged and requiring no counterfactual modeling. On standard continuous-control benchmarks, EWA-VQ-ODT improves average evaluation return and sample efficiency over an Online Decision Transformer (ODT) baseline. The module is lightweight, easy to integrate, and interpretable through per-code attraction traces. In addition, we provide two concise theoretical analyses that formalize the attraction dynamics and bound the effect of the bias on attention, supporting safe and stable integration into transformer policies.

1 Introduction

Transformers have become a compelling substrate for control by casting reinforcement learning as sequence modeling over trajectories [11, 21, 31]. In this view, credit is propagated through content-based attention: tokens (returns, states, actions) attend to one another to predict the next action. While effective and scalable, standard attention lacks an *explicit* memory of action-specific outcomes—there is no persistent state that encodes, in compact form, which kinds of actions have recently “worked” or “failed.” As a result, transformer policies may repeatedly rediscover local regularities through attention rather than carrying forward a simple, actionable summary of past success and failure associated with actions.

Behavioral and cognitive theories suggest a complementary inductive bias. Experience-Weighted Attraction (EWA) [8] and related cognitive architectures (e.g., ACT-R; 1) posit that humans maintain per-action “mental accounts” that accumulate outcomes with decay: attractions persist for all actions, decrease when unchosen, and increase after rewarding choices. Such accounts bias future selection toward historically effective actions without requiring counterfactual modeling. Bringing this idea to transformer policies suggests endowing action tokens with a lightweight, explicit memory of outcomes that persists across timesteps and modulates attention.

This paper. We propose **EWA-VQ-ODT**, a cognitively inspired add-on for Decision/Online Decision Transformers that maintains a per-action (code) attraction and injects it into attention. Continuous actions are routed to a compact set of *vector-quantized* surrogates via direct grid lookup; each code stores a scalar attraction updated online by a simplified EWA rule (decay every step; additive reinforcement on the chosen code with signed reward). We then add a small bias to the pre-softmax attention *columns* corresponding to action tokens, so tokens associated with high (low)

attraction receive slightly more (less) attention. The backbone architecture, objective, and training loop remain unchanged.

Two design choices make the module practical at scale. First, we adopt an EWA-inspired update without experience normalization or counterfactual payoffs, preserving the recency signal while keeping computation strictly online. Second, we discretize actions with a fixed codebook and constant-time routing, avoiding nearest-neighbor scans and enabling vectorized GPU updates.

Positioning. Our approach complements prior strands in three areas. (i) *Transformers for RL*: beyond return-conditioning and stability improvements [11, 25, 31], we introduce an explicit, action-specific memory that biases attention with minimal overhead, orthogonal to value-regularization or architectural changes. (ii) *Cognitively grounded RL*: rather than learning external preference models or episodic stores [4, 27], we integrate an EWA-style mental account directly into attention, providing an interpretable, persistent prior over action relevance. (iii) *Action representation*: compared to learned, state-conditioned discretization [23] or kernel-based soft assignments, our fixed VQ routing trades some flexibility for speed and stability while exposing a small set of interpretable action surrogates whose attractions can be monitored.

Empirical and theoretical preview. On D4RL continuous-control tasks, EWA-VQ-ODT improves average evaluation return over an identically configured ODT baseline, with gains that tend to appear *early* in training (better sample efficiency) and reduced variability in some settings. Theoretically, we provide two small guarantees that justify the design and its tuning: (i) a closed-form expression for the attraction process together with boundedness and an expected steady-state under stationarity, which formalizes the “mental account” as a stable exponentially-weighted sum; and (ii) a softmax perturbation bound showing that clipping the bias to $\|\beta A\|_\infty \leq \varepsilon$ limits the attention drift to at most $\tanh(\varepsilon)$ in total variation, supporting safe bias scaling. Both proofs are elementary and included in the appendix.

Contributions.

- We introduce Experience-Weighted Attraction with Vector Quantization for Online Decision Transformers (EWA-VQ-ODT), a drop-in module that maintains per-action (code) attractions online and injects them as a small bias on action-token attention columns in DT/ODT.
- We instantiate a fast vector-quantized routing scheme for continuous actions (fixed codebook, direct grid lookup) enabling constant-time mapping and batched GPU updates.
- We provide empirical improvements in average return and sample efficiency on standard continuous-control tasks, and two concise theoretical analyses that (a) characterize attraction dynamics and (b) bound the impact of the bias on attention.

2 Related Work

2.1 Transformers for Reinforcement Learning

A growing line of work recasts reinforcement learning as sequence modeling. Early evidence showed that high-capacity sequence predictors can model trajectories and even support planning with generic decoding procedures [21]. Building on this view, Chen et al. [11] introduced the *Decision Transformer* (DT), which conditions a causal Transformer on desired return, past states, and actions to generate actions autoregressively, matching strong offline RL baselines without explicit value functions. Zheng et al. [31] extended this paradigm to interactive settings with the *Online Decision Transformer* (ODT), blending offline pretraining and online finetuning via sequence-level entropy regularization for sample-efficient exploration.

Applying Transformers in RL also raises stability and credit-assignment challenges. Parisotto et al. [25] identified optimization issues of standard Transformers under RL objectives and proposed GTrXL, architectural modifications that improve stability and performance across memory-intensive tasks. Complementary advances address long-horizon credit propagation; for example, Malkin et al. [24] introduced trajectory balance objectives in GFlowNets to propagate credit more efficiently along action sequences. Other works combine sequence modeling with value-based structure: Hu et al. [19]

regularize the Transformer with learned Q -values to better stitch optimal segments from suboptimal data in offline RL.

Scaling trends in control further support the utility of Transformer policies. RT-1 [7] trains a robotics Transformer on large, diverse real-world data and demonstrates broad generalization across tasks, indicating that high-capacity sequence models can absorb heterogeneous experience in control domains.

Our work targets a complementary axis: *explicit, action-specific memory*. Rather than modifying architectures for stability [25] or augmenting objectives with values [19], we inject a lightweight, EWA-inspired per-action attraction signal into attention columns of DT/ODT. This preserves the return-conditioning benefits of sequence modeling [11, 31] while providing a persistent, decaying “mental account” of action outcomes that improves credit flow with minimal overhead and no counterfactual modeling.

2.2 Cognitively Grounded RL and Experience-Weighted Attraction

Behavioral and cognitive theories provide formal accounts of how humans accumulate and use action-specific experience. Camerer [8] synthesizes extensive experimental evidence into behavioral game theory, emphasizing learning dynamics and limited strategic depth; a central component is *experience-weighted attraction* (EWA), where per-action “attractions” aggregate outcomes with depreciation, predicting regularities in repeated interactive settings. Cognitive architectures such as ACT-R [1] instantiate persistent declarative and procedural memories with activation and decay, while instance-based learning (IBL) [16] explains dynamic choice by retrieving and blending past situation-action-utility instances with recency and utility weighting. These strands motivate maintaining explicit, decaying, action-specific memories—the conceptual basis for our “mental accounts” of success and failure.

Cognitively informed RL has also been explored in decision settings that require interpretability and robustness, including human-AI teaming for security. Cognitive-hierarchy models of bounded rationality [10] and prospect-theoretic accounts of subjective valuation under uncertainty [22] have been instantiated in deep RL for cloud defense and analyzed for their impact on policy learning [2, 3]. In web-based human-in-the-loop DRL experiments on Amazon Mechanical Turk, Aref et al. [3] report decision patterns consistent with Prospect Theory (PT) and Cumulative Prospect Theory (CPT): participants tend to avoid re-selection after failure but persist after success, reflecting heightened loss sensitivity and biased probability weighting (underestimating gains after loss and overestimating continued success). This asymmetry is also aligned with EWA’s mechanism—decaying attractions for unchosen/unsuccessful actions and reinforcement of attractions after successful choices. Collectively, these studies underscore the value of explicit, human-interpretable biases and persistent memory mechanisms when actions must be justified and robust to nonstationarity.

Within RL, several mechanisms echo these principles by adding fast memory or sharpening credit assignment. Neural Episodic Control [27] augments deep agents with a semi-tabular episodic store for rapidly updated value estimates, enabling swift assimilation of new experience under sparse reward. RUDDER [4] tackles delayed credit by redistributing returns and decomposing contributions, converting long-horizon credit into a supervised regression problem. Resource-rational RL [6] models human exploration as bounded optimality under computational costs, supporting lightweight memory and priors as plausible inductive biases. In complementary directions, preference-based RL and human feedback pipelines—from policy shaping with direct human labels [17] and learning from pairwise preferences [12] to large-scale RLHF [5], fine-grained segment-level rewards [30], few-shot preference models [18], and personalized latent preference learning [26]—demonstrate that simple, interpretable supervisory signals can reliably steer complex policies. Robust adversarial model-based offline RL [28] addresses stability under distribution shift and is orthogonal to cognitive inductive biases. Closer to sequence models, hierarchical/in-context formulations [20] reduce decision horizons by structuring control, suggesting a productive interface between cognitive principles and transformers.

Our contribution differs in both target and mechanism. Rather than learning external reward models or episodic memories, we integrate a *per-action, decaying attraction* directly into the transformer’s attention via a small bias on action-token columns. This EWA-inspired signal is vector-quantized for continuous actions, learned online from observed rewards (positive and negative), and requires

no counterfactual modeling or architectural changes. In effect, we endow Decision/Online Decision Transformers with an explicit, interpretable memory of action outcomes that persistently shapes credit flow, complementing prior cognitive and human–feedback approaches.

2.3 Action Representation and Quantization

Representing continuous actions with compact discrete surrogates can simplify learning, improve stability, and enable structured inductive biases. A prominent line of work learns discrete latent codes with vector quantization. van den Oord et al. [29] introduced VQ–VAE, which pairs an autoregressive prior with a learned codebook to obtain discrete latents that avoid posterior collapse and support high-fidelity generation. Building on this idea for control, Luo et al. [23] propose state-conditioned action quantization via VQ–VAE in offline RL, showing that discretizing the action space allows conservative objectives (e.g., IQL, CQL, BRAC) to be applied more precisely, yielding substantial gains on robotics benchmarks.

Beyond direct action discretization, several strands learn *action abstractions* or latent representations that can serve as surrogates. Unsupervised skill discovery methods such as DIAYN [14] maximize diversity to produce a repertoire of discrete skills that can be composed or fine-tuned for downstream tasks, effectively quantizing behavior space. Latent state-space approaches, e.g., DeepMDP [15], learn compact predictive representations that facilitate planning and policy learning; although continuous, these latents can be clustered or discretized when needed to provide decision tokens.

Kernelized clustering provides an alternative, *soft* form of quantization in which assignments are driven by similarities in a reproducing kernel space. Kernel k –means and its spectral counterparts [13] (and soft variants via fuzzy clustering) enable similarity-aware grouping without explicit parametric codebooks, at the cost of potentially higher per-step assignment overhead.

We target a lightweight, *online* surrogate representation compatible with transformer policies. Instead of learning a state-conditioned codebook [23] or maintaining kernel similarities [13], we use a fixed, grid-derived codebook with direct lookup to achieve constant-time routing. On these discrete surrogates, we maintain an EWA-style per-action memory and inject it as a small bias into attention. This retains many practical benefits of discretization (stable routing, small memory) while enabling an explicit, cognitively motivated mechanism for credit flow over actions.

3 Preliminaries

Reinforcement learning. We consider a Markov decision process (MDP) $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$. At time t , the agent observes $s_t \in \mathcal{S}$, chooses $a_t \in \mathcal{A}$ via policy $\pi(a_t | s_t)$, receives $r_t = r(s_t, a_t)$, and transitions to $s_{t+1} \sim P(\cdot | s_t, a_t)$. The objective is to maximize expected discounted return $\mathbb{E}_\pi[\sum_{k=0}^{\infty} \gamma^k r_{t+k}]$.

Decision Transformers. Decision Transformers (DT; Chen et al. [11]) recast policy learning as sequence modeling over trajectories $\tau = (R_1, s_1, a_1, \dots, R_T, s_T, a_T)$, where $R_t = \sum_{k=t}^T \gamma^{k-t} r_k$ is return-to-go. Tokens (returns, states, actions) are embedded into a common space and processed by a causal transformer; with token embedding x_t , queries/keys/values are $Q_t = x_t W^Q$, $K_t = x_t W^K$, $V_t = x_t W^V$. DT predicts a_t autoregressively and is trained by maximum likelihood:

$$\mathcal{L}_{\text{DT}} = - \sum_{t=1}^T \log p_\theta(a_t | \tau_{\leq t}). \quad (1)$$

Online Decision Transformer. Online Decision Transformer (ODT; Zheng et al. [31]) adapts DT to interactive data collection by training a return-conditioned stochastic policy $\pi_\theta(a_t | s_{\leq t}, g_{\leq t})$, where g_t denotes return-to-go. Learning minimizes the negative log-likelihood (optionally with sequence-level entropy regularization); see Zheng et al. [31] for details.

Experience-Weighted Attraction (EWA). EWA [9] maintains, for each action j , an *attraction* $A_j(t)$ that accumulates payoff evidence with decay. The canonical update is

$$A_j(t) = \frac{(1 - \phi) N(t-1) A_j(t-1) + [\delta + (1 - \delta) \mathbf{1}\{j = j_t\}] \pi_j(t)}{N(t)}, \quad N(t) = (1 - \rho) N(t-1) + 1, \quad (2)$$

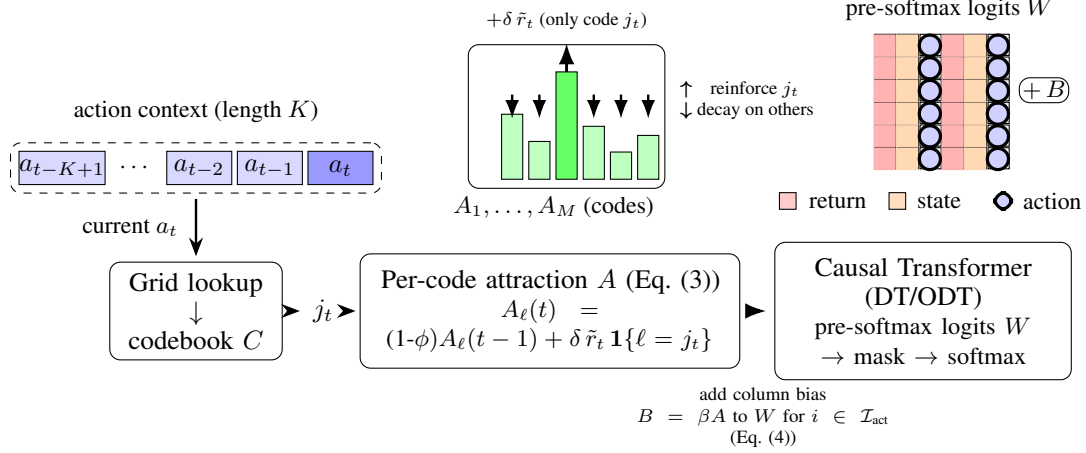


Figure 1: **EWA-VQ-ODT overview.** (A) The last K actions form the context; the current action a_t is routed by a grid lookup (cell-code table T) to code j_t . (B) Per-code attractions implement a recency-weighted memory via decay $(1 - \phi)$ and reinforcement $\delta \tilde{r}_t$ on the chosen code (Eq. (3)). (C) Attractions add a pre-softmax *column bias* $B = \beta A$ to the action-token *columns* of the attention logits W (Eq. (4)); highlighted heatmap columns indicate $i \in \mathcal{I}_{\text{act}}$.

where $\mathbf{1}\{j = j_t\}$ indicates whether j was chosen at t , $\pi_j(t)$ is the (possibly hypothetical) payoff assigned to j , $\phi \in (0, 1)$ depreciates attractions, $\rho \in (0, 1)$ depreciates experience, and $\delta \in [0, 1]$ balances realized vs. foregone payoffs. Attractions for *all* actions persist and decay even when unchosen.

4 Method

Overview. Figure 1 summarizes the proposed EWA-VQ-ODT framework. **(A)** A length- K action context is maintained, and the current action a_t is routed by a grid lookup to a discrete code j_t in a fixed vector-quantized codebook. **(B)** Each code ℓ holds an attraction $A_\ell(t)$ updated online by a simplified EWA rule with global decay $(1 - \phi)$ and reinforcement $\delta \tilde{r}_t$ *only* for the chosen code j_t (Eq. (3)); the up arrow marks reinforcement for j_t while dashed down arrows indicate decay on others. **(C)** The attraction vector A is scaled to a column bias $B = \beta A$ and added to the pre-softmax attention logits *only* on action-token columns (Eq. (4)), gently steering attention toward actions with higher recent attraction without changing the backbone.

4.1 Experience-Weighted Attraction (EWA) for Decision Transformers

We endow a Decision/Online Decision Transformer with an explicit, per-action-surrogate memory by maintaining a scalar *attraction* for each vector-quantized code (Sec. 4.2) and updating it online. Let j_t be the routed code for action a_t . Our EWA-inspired update is

$$A_\ell(t) = (1 - \phi) A_\ell(t-1) + \delta \tilde{r}_t \mathbf{1}\{\ell = j_t\}, \quad (3)$$

with $\phi \in (0, 1)$ (forgetting), $\delta \in [0, 1]$ (chosen-action weight), and $\tilde{r}_t$ a centered/clipped reward so that positive outcomes increase attraction (“success”) and negative outcomes decrease it (“failure”). Thus, previously observed but unchosen codes retain a decaying memory; the chosen code receives the incremental reinforcement. Unseen codes remain at zero until first routed. The resulting $A_{j_t}(t)$ is later injected as a small column-bias into action-token attention logits (Eq. (4)).

Why the simplified form? The canonical EWA (Sec. 3) normalizes by $N(t)$ and can include counterfactual payoffs. We omit both for (i) *compatibility* with online sequence models (no counterfactual model required), (ii) *stability/efficiency* (constant-time, vectorized updates without a running denominator), and (iii) *calibration* (the attention bias absorbs scale). Empirically, Eq. (3) preserves the intended recency-weighted reinforcement signal while keeping the mechanism lightweight and easy to integrate.

Proposition 4.1 (Closed form, boundedness, steady state). *Let the per-code attraction follow the simplified EWA update*

$$A_\ell(t) = (1 - \phi)A_\ell(t-1) + \delta \tilde{r}_t \mathbf{1}\{\ell = j_t\}, \quad \phi \in (0, 1), \quad |\tilde{r}_t| \leq R.$$

Then $A_\ell(t)$ admits a closed form, is uniformly bounded by $\delta R/\phi$, and under stationarity satisfies

$$\lim_{t \rightarrow \infty} \mathbb{E}[A_\ell(t)] = \frac{\delta p_\ell \mu_\ell}{\phi}.$$

Proof sketch. Unrolling the recursion yields the closed form and boundedness via a geometric sum. Under stationarity, the expectation satisfies a linear recursion with fixed point $(\delta p_\ell \mu_\ell)/\phi$. Full details are deferred to Appendix A.1. $\square$

The proposition 4.1 implies that $A_\ell(t)$ is an exponentially weighted sum of realized rewards for code ℓ , with weight $(1 - \phi)^{t-\tau}$ on the outcome at time τ ; older events thus contribute less. With zero initialization, the attraction is uniformly bounded as $|A_\ell(t)| \leq \frac{\delta R}{\phi}$. Under stationarity with $\Pr(j_t = \ell) = p_\ell$ and $\mathbb{E}[\tilde{r}_t | j_t = \ell] = \mu_\ell$, the long-run expectation converges as $\lim_{t \rightarrow \infty} \mathbb{E}[A_\ell(t)] = \frac{\delta p_\ell \mu_\ell}{\phi}$, increasing with δ and $p_\ell \mu_\ell$ and decreasing with the forgetting rate ϕ .

4.2 Vector-Quantized Action Surrogates with Direct Grid Lookup

Continuous actions $a_t \in [-1, 1]^d$ are routed to M discrete *codes* $C = \{c_i\}_{i=1}^M$ via vector quantization. A fixed codebook is built once from an axis-aligned grid with b bins per dimension; the grid table $T : \{0, \dots, b^d - 1\} \rightarrow \{1, \dots, M\}$ maps each cell to its nearest code. At runtime, a_t is quantized to a cell and the code index is read from T : $j_t = T(\text{cell}(a_t))$. Misses (rare) fall back to a small batched L_2 search. The online and offline procedures appear in Alg. 1 and Alg. 2.

Algorithm 1 EWAVQ (Vector-Quantized EWA), per trajectory

Input: Actions $\{a_t\}_{t=1}^K$, rewards $\{r_t\}_{t=1}^K$, decay ϕ , weight δ , codebook $C = \{c_i\}_{i=0}^{N-1}$, grid bins b , table T

Output: D_{codebook} , $D_{\text{trajectory}}$

$A \in \mathbb{R}^N \leftarrow \mathbf{0}$; $D_{\text{trajectory}} \leftarrow []$

Function ROUTE(a)

 Map $a \in [-1, 1]^D$ to grid coords q and linear index ℓ ; **if** ℓ is cached in T **then**
 | **return** $T[\ell]$

end

else

 | **return** $\arg \min_{i \in \{0, \dots, N-1\}} \|a - c_i\|_2$

end

end

for $t \leftarrow 1$ **to** K **do**

$i^* \leftarrow \text{ROUTE}(a_t)$ $A \leftarrow (1 - \phi)A$ // global decay (vectorized)

$A[i^*] \leftarrow A[i^*] + \delta r_t$ **append** $(a_t, r_t, i^*, A[i^*])$ to $D_{\text{trajectory}}$

end

$D_{\text{codebook}} \leftarrow \{(i, A[i]) \mid A[i] \neq 0\}$ **return** D_{codebook} , $D_{\text{trajectory}}$

Algorithm 2 Grid-based Action Quantization (cell $\rightarrow$ code)

Input: Dim D , requested codes N_{req} , bins b_{req}

Output: $T \in \{0, \dots, N-1\}^{b^D}$

$b \leftarrow \min\{8, \max\{2, \lfloor N_{\text{req}}^{1/D} \rfloor\}\}$; **while** $b^D > N_{\text{req}}$ **and** $b > 2$ **do**
| $b \leftarrow b - 1$

end

if $D \geq 6$ **and** $b^D > N_{\text{req}}$ **then**

| $N \leftarrow \min(b^D, 128)$

end

else

| $N \leftarrow N_{\text{req}}$

end

for $i \leftarrow 0$ **to** $N - 1$ **do**

| Let $g_i \in \{0, \dots, b-1\}^D$ be the base- b digits of i ; Set code $c_i \in [-1, 1]^D$ by $c_i[d] \leftarrow \frac{2g_i[d]}{b-1} - 1$
| for $d = 1..D$

end

Allocate T of size b^D ; **for** cells $\ell \leftarrow 0$ **to** $b^D - 1$ **in batches do**

| Map ℓ to g_ℓ (base- b) and $a_\ell[d] \leftarrow \frac{2g_\ell[d]}{b-1} - 1$; $T[\ell] \leftarrow \arg \min_{i \in \{0, \dots, N-1\}} \|a_\ell - c_i\|_2$
| // vectorized on GPU

end

Cache T under key (D, b, N, env) and return

4.3 Injecting Attractions into Attention

Let $W \in \mathbb{R}^{N \times H \times S \times S}$ denote pre-softmax attention logits over tokenized returns, states, and actions. Let $\mathcal{I}_{\text{act}} \subset \{1, \dots, S\}$ be the action-token column indices and let $t(i)$ be the timestep associated with action column i . From (3), define the per-batch, per-head *attraction bias* as

$$B_{n,h,:i} = \begin{cases} \beta A_{j_{t(i)}}(t(i)) \mathbf{1}_S, & i \in \mathcal{I}_{\text{act}}, \\ \mathbf{0}_S, & \text{otherwise,} \end{cases} \quad \tilde{W} = W + B, \quad (4)$$

where $\mathbf{1}_S$ is a length- S all-ones vector (broadcast along rows) and $\beta > 0$ is a small scalar (or per-head scale; see ablations). We then apply the usual causal/padding masks and softmax. Intuitively, $\beta A_{j_{t(i)}}$ is a column-bias that increases (decreases) attention mass toward action tokens with stronger (weaker) accumulated attraction—an explicit, recency-weighted memory of success and failure.¹

Lemma 4.2 (Controlled attention drift under a bounded bias). *Let $p = \text{softmax}(z) \in \Delta^n$ and $q = \text{softmax}(z + b)$ for any bias b with $\|b\|_\infty \leq \varepsilon$. Then the total variation distance satisfies*

$$\|q - p\|_{\text{TV}} \leq \tanh(\varepsilon).$$

Proof sketch. The likelihood ratio q_i/p_i is bounded in $[e^{-2\varepsilon}, e^{2\varepsilon}]$, which implies $\|q - p\|_{\text{TV}} \leq \tanh(\varepsilon)$. Full details are in Appendix A.2. $\square$

Intuitively, perturbing logits by at most $\pm\varepsilon$ constrains each attention probability’s multiplicative change to $[e^{-2\varepsilon}, e^{2\varepsilon}]$; the worst case occurs when some entries get the full boost and others the full suppression, yielding the $\tanh(\varepsilon)$ bound. For small ε , $\tanh(\varepsilon) \approx \varepsilon$, so clipping $\|\beta A\|_\infty \leq \varepsilon$ provides a simple, interpretable cap on attention drift.

¹We keep the attention architecture unchanged: the bias is additive, applied before masking/softmax.

Table 1: EWA-VQ-ODT vs. ODT on walker2d-medium-replay-v2. Averages are over seeds 1–3 and training steps (gm = geometric mean). A ✓ indicates that the change is desirable.

Metric	EWA-VQ-ODT Avg	ODT Avg	Δ vs. ODT (%)
evaluation/return_mean_gm	167.1372	82.3904	+102.9% ✓
evaluation/return_std_gm	155.3095	57.3895	+170.6% ×
evaluation/return_vs_samples	179.2537	123.3744	+45.3% ✓
evaluation/length_mean_gm	134.5278	87.4389	+53.9% ✓
evaluation/length_std_gm	70.0563	16.2292	+331.7% ×
aug_traj/return	87.5206	64.0198	+36.7% ✓
aug_traj/length	116.0667	90.4667	+28.3% ✓

5 Experiments

5.1 Setup

Environments. We evaluate on two D4RL continuous-control benchmarks: hopper-medium-v2 (action dim. $d=3$) and walker2d-medium-replay-v2 ($d=6$). Datasets contain suboptimal offline trajectories; we fine-tune online following the standard offline-to-online protocol.

Baselines. We compare EWA-VQ-ODT to the **Online Decision Transformer (ODT)**. Architectures (backbone width/depth), tokenization, context length, optimizer, and training schedules are matched.

Training configuration. Unless noted: 10 online iterations with evaluation every two iterations; batch size 64; context length $K=20$; Adam ($\text{lr}=10^{-4}$) with linear warmup; 10 evaluation episodes per checkpoint; 3 random seeds.

EWA-VQ-ODT hyperparameters. Attention bias scale $\beta=0.05$; decay $\phi=0.05$; chosen weight $\delta=0.8$; codebook size $M=27$; grid bins chosen adaptively per dimension. (Ablations in the Appendix.)

Evaluation protocol. At each checkpoint we compute per-seed metrics; we then average across seeds at each step, and finally average those per-step means across all training steps to obtain a single scalar per (environment, algorithm, metric). Improvements are reported as relative change vs. ODT.

5.2 Metrics

We report:

- evaluation/return_mean_gm: geometric mean of evaluation returns
- evaluation/return_std_gm: dispersion of evaluation returns
- evaluation/return_vs_samples: return as a function of consumed samples
- evaluation/length_mean_gm: geometric mean of evaluation episode lengths
- evaluation/length_std_gm: dispersion of episode lengths
- aug_traj/return: return of trajectories generated during fine-tuning
- aug_traj/length: length of trajectories generated during fine-tuning

5.3 Main Results

Findings. On walker2d-medium-replay-v2, EWA-VQ-ODT more than doubles average evaluation return and improves length-based metrics (✓), but also produces larger dispersion in both return and episode length (×), consistent with faster early gains and higher mid-training volatility. On hopper-medium-v2, EWA-VQ-ODT consistently improves average return and episode length (✓) while *reducing* dispersion (✓), indicating more stable policies. Across both tasks, evaluation/return_vs_samples is higher (✓), suggesting better sample efficiency.

Table 2: EWA-VQ-ODT vs. ODT on hopper-medium-v2. Averages are over seeds 1–3 and training steps (gm = geometric mean). A ✓ indicates that the change is desirable.

Metric	EWA-VQ-ODT Avg	ODT Avg	Δ vs. ODT (%)
evaluation/return_mean_gm	1343.8541	1254.8157	+7.1% ✓
evaluation/return_std_gm	307.9207	403.2548	−23.6% ✓
evaluation/return_vs_samples	1325.0530	1222.1248	+8.4% ✓
evaluation/length_mean_gm	680.5667	629.0445	+8.2% ✓
evaluation/length_std_gm	162.3041	187.2219	−13.3% ✓
aug_traj/return	867.9049	884.5185	−1.9% ×
aug_traj/length	357.5000	376.7667	−5.1% ×

Discussion. These results support the hypothesis that an explicit, recency-weighted per-action memory acts as a useful inductive bias for transformer-based control: it accelerates learning and, in lower-dimensional action spaces (Hopper), also stabilizes outcomes. In higher-dimensional settings (Walker2d), the same mechanism can over-reinforce frequent codes without additional regularization, motivating *per-head* learned bias scales, reward centering/clipping in the attraction update, and modest adjustments to codebook granularity. We report ablations on β , ϕ , M , and reward normalization, plus additional seeds and D4RL-normalized scores, in the Appendix.

Compute and overhead. EWA-VQ-ODT adds a small GPU-resident state (code attractions) and a column-wise bias to attention logits. Routing is $O(d)$ via direct grid lookup; updates are vectorized indexed adds. In our runs this incurred negligible training-time overhead relative to ODT.

6 Conclusion

We introduced **EWA-VQ-ODT**, a minimal, cognitively inspired add-on that equips Decision/Online Decision Transformers with an explicit per-action outcome memory. Continuous actions are routed to a compact vector-quantized codebook via direct grid lookup; each code maintains a decayed attraction updated online by a simplified EWA rule. We inject these attractions as a small bias on action-token attention columns, leaving the backbone, objective, and training loop unchanged. On standard continuous-control benchmarks, EWA-VQ-ODT improves average return and sample efficiency over a strong ODT baseline, while adding negligible computational overhead and offering interpretable diagnostics via attraction traces.

Beyond empirical gains, we provided two simple theoretical analyses: a closed-form characterization of the attraction process establishing boundedness and an expected steady state, and a softmax perturbation bound that constrains attention drift when the bias is clipped. These results justify stable scaling of the decay/bias parameters and support safe integration into transformer attention.

Limitations. Gains are environment-dependent: higher-dimensional action spaces can exhibit larger mid-training variance or late-training regressions if the bias scale or codebook granularity are mis-tuned. A fixed codebook trades similarity generalization for routing speed, and hard assignments introduce quantization error. Our study focuses on simulator benchmarks; human-subject evaluations and broader domains are left for future work.

Outlook. Promising directions include (i) per-head or token-wise learned bias scales and normalization of the attraction signal; (ii) learned or hierarchical codebooks and soft/state-conditioned routing; (iii) coupling EWA-style memory with value- or model-based credit assignment; and (iv) expanded evaluation (more seeds, tasks, and real-world settings). Overall, explicit, low-dimensional memories of action “success” and “failure” provide a practical inductive bias for transformer-based RL, complementing content-based attention without complicating the architecture.

References

- [1] John R Anderson. *How can the human mind occur in the physical universe?* Oxford University Press, 2009.

- [2] Zahra Aref and Narayan B Mandayam. Impact of subjectivity in deep reinforcement learning based defense of cloud storage. In *IEEE INFOCOM 2022-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, pages 1–6. IEEE, 2022.
- [3] Zahra Aref, Sheng Wei, and Narayan B Mandayam. Human-ai collaboration in cloud security: Cognitive hierarchy-driven deep reinforcement learning. *arXiv preprint arXiv:2502.16054*, 2025.
- [4] Jose A Arjona-Medina, Michael Gillhofer, Michael Widrich, Thomas Unterthiner, Johannes Brandstetter, and Sepp Hochreiter. Rudder: Return decomposition for delayed rewards. *Advances in Neural Information Processing Systems*, 32, 2019.
- [5] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022.
- [6] Marcel Binz and Eric Schulz. Modeling human exploration through resource-rational reinforcement learning. *Advances in neural information processing systems*, 35:31755–31768, 2022.
- [7] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.
- [8] Colin Camerer. *Behavioral game theory: Experiments in strategic interaction*. Princeton university press, 2003.
- [9] Colin Camerer and Teck Hua Ho. Experience-weighted attraction learning in normal form games. *Econometrica*, 67(4):827–874, 1999.
- [10] Colin F Camerer, Teck-Hua Ho, and Juin-Kuan Chong. A cognitive hierarchy model of games. *The quarterly journal of economics*, 119(3):861–898, 2004.
- [11] Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Misha Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision transformer: Reinforcement learning via sequence modeling. *Advances in neural information processing systems*, 34:15084–15097, 2021.
- [12] Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30, 2017.
- [13] Inderjit S Dhillon, Yuqiang Guan, and Brian Kulis. Kernel k-means: spectral clustering and normalized cuts. In *Proceedings of the tenth ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 551–556, 2004.
- [14] Benjamin Eysenbach, Abhishek Gupta, Julian Ibarz, and Sergey Levine. Diversity is all you need: Learning skills without a reward function. *arXiv preprint arXiv:1802.06070*, 2018.
- [15] Carles Gelada, Saurabh Kumar, Jacob Buckman, Ofir Nachum, and Marc G Bellemare. Deep-mdp: Learning continuous latent space models for representation learning. In *International conference on machine learning*, pages 2170–2179. PMLR, 2019.
- [16] Cleotilde Gonzalez, Javier F Lerch, and Christian Lebiere. Instance-based learning in dynamic decision making. *Cognitive Science*, 27(4):591–635, 2003.
- [17] Shane Griffith, Kaushik Subramanian, Jonathan Scholz, Charles L Isbell, and Andrea L Thomaz. Policy shaping: Integrating human feedback with reinforcement learning. *Advances in neural information processing systems*, 26, 2013.
- [18] Donald Joseph Hejna III and Dorsa Sadigh. Few-shot preference learning for human-in-the-loop rl. In *Conference on Robot Learning*, pages 2014–2025. PMLR, 2023.

- [19] Shengchao Hu, Ziqing Fan, Chaoqin Huang, Li Shen, Ya Zhang, Yanfeng Wang, and Dacheng Tao. Q-value regularized transformer for offline reinforcement learning. *arXiv preprint arXiv:2405.17098*, 2024.
- [20] Sili Huang, Jifeng Hu, Hechang Chen, Lichao Sun, and Bo Yang. In-context decision transformer: Reinforcement learning via hierarchical chain-of-thought. *arXiv preprint arXiv:2405.20692*, 2024.
- [21] Michael Janner, Qiyang Li, and Sergey Levine. Offline reinforcement learning as one big sequence modeling problem. *Advances in neural information processing systems*, 34:1273–1286, 2021.
- [22] Daniel Kahneman and Amos Tversky. Prospect theory: An analysis of decision under risk. In *Handbook of the fundamentals of financial decision making: Part I*, pages 99–127. World Scientific, 2013.
- [23] Jianlan Luo, Perry Dong, Jeffrey Wu, Aviral Kumar, Xinyang Geng, and Sergey Levine. Action-quantized offline reinforcement learning for robotic skill learning. In *Conference on Robot Learning*, pages 1348–1361. PMLR, 2023.
- [24] Nikolay Malkin, Moksh Jain, Emmanuel Bengio, Chen Sun, and Yoshua Bengio. Trajectory balance: Improved credit assignment in gflowtnets. *Advances in Neural Information Processing Systems*, 35:5955–5967, 2022.
- [25] Emilio Parisotto, Francis Song, Jack Rae, Razvan Pascanu, Caglar Gulcehre, Siddhant Jayakumar, Max Jaderberg, Raphael Lopez Kaufman, Aidan Clark, Seb Noury, et al. Stabilizing transformers for reinforcement learning. In *International conference on machine learning*, pages 7487–7498. PMLR, 2020.
- [26] Sriyash Poddar, Yanming Wan, Hamish Ivison, Abhishek Gupta, and Natasha Jaques. Personalizing reinforcement learning from human feedback with variational preference learning. *Advances in Neural Information Processing Systems*, 37:52516–52544, 2024.
- [27] Alexander Pritzel, Benigno Uribe, Sriram Srinivasan, Adria Puigdomenech Badia, Oriol Vinyals, Demis Hassabis, Daan Wierstra, and Charles Blundell. Neural episodic control. In *International conference on machine learning*, pages 2827–2836. PMLR, 2017.
- [28] Marc Rigter, Bruno Lacerda, and Nick Hawes. Rambo-rl: Robust adversarial model-based offline reinforcement learning. *Advances in neural information processing systems*, 35:16082–16097, 2022.
- [29] Aäron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. In *Neural Information Processing Systems*, 2017. URL <https://api.semanticscholar.org/CorpusID:20282961>.
- [30] Zeqiu Wu, Yushi Hu, Weijia Shi, Nouha Dziri, Alane Suhr, Prithviraj Ammanabrolu, Noah A Smith, Mari Ostendorf, and Hannaneh Hajishirzi. Fine-grained human feedback gives better rewards for language model training. *Advances in Neural Information Processing Systems*, 36: 59008–59033, 2023.
- [31] Qinqing Zheng, Amy Zhang, and Aditya Grover. Online decision transformer. In *international conference on machine learning*, pages 27042–27059. PMLR, 2022.

A Proofs of Theoretical Results

A.1 Proof of Proposition 4.1

Proposition 4.1 [Closed form, boundedness, steady state]

Let the per-code attraction follow the simplified EWA update

$$A_\ell(t) = (1 - \phi)A_\ell(t-1) + \delta \tilde{r}_t \mathbf{1}\{\ell = j_t\}, \quad \phi \in (0, 1), \quad |\tilde{r}_t| \leq R.$$

Then $A_\ell(t)$ admits a closed form, is uniformly bounded by $\delta R/\phi$, and under stationarity satisfies

$$\lim_{t \rightarrow \infty} \mathbb{E}[A_\ell(t)] = \frac{\delta p_\ell \mu_\ell}{\phi}.$$

Proof. As described in the main text, $A_\ell(t)$: the “attraction” (memory) for action code ℓ after step t . Update rule (simplified EWA) is:

$$A_\ell(t) = (1 - \phi) A_\ell(t-1) + \delta \tilde{r}_t \mathbf{1}\{\ell = j_t\},$$

where $0 < \phi < 1$ is the decay rate, δ scales new rewards, $\tilde{r}_t$ is bounded with $|\tilde{r}_t| \leq R$, j_t is the code chosen at time t , and $\mathbf{1}\{\ell = j_t\}$ is the indicator of choosing ℓ .

Every step shrinks all memories by $(1 - \phi)$ and adds $\delta \tilde{r}_t$ only to the chosen code’s memory.

Step 1: Closed form

Unrolling the recursion gives:

$$\begin{aligned} A_\ell(t) &= (1 - \phi)A_\ell(t-1) + \delta \tilde{r}_t \mathbf{1}\{\ell = j_t\} \\ &= (1 - \phi) \left[(1 - \phi)A_\ell(t-2) + \delta \tilde{r}_{t-1} \mathbf{1}\{\ell = j_{t-1}\} \right] + \delta \tilde{r}_t \mathbf{1}\{\ell = j_t\} \\ &= (1 - \phi)^2 A_\ell(t-2) + (1 - \phi) \delta \tilde{r}_{t-1} \mathbf{1}\{\ell = j_{t-1}\} + \delta \tilde{r}_t \mathbf{1}\{\ell = j_t\} \\ &\vdots \\ &= (1 - \phi)^t A_\ell(0) + \delta \sum_{\tau=1}^t (1 - \phi)^{t-\tau} \tilde{r}_\tau \mathbf{1}\{\ell = j_\tau\}. \end{aligned}$$

It demonstrates that $A_\ell(t)$ is a geometrically decayed sum of past realized rewards for code ℓ .

Step 2: Uniform boundedness

We start from the unrolled form, which is already established:

$$A_\ell(t) = (1 - \phi)^t A_\ell(0) + \delta \sum_{\tau=1}^t (1 - \phi)^{t-\tau} \tilde{r}_\tau \mathbf{1}\{\ell = j_\tau\}.$$

First, apply the triangle inequality and absolute values to separate the two terms:

$$|A_\ell(t)| \leq |(1 - \phi)^t A_\ell(0)| + \delta \left| \sum_{\tau=1}^t (1 - \phi)^{t-\tau} \tilde{r}_\tau \mathbf{1}\{\ell = j_\tau\} \right|.$$

This step follows because $|x + y| \leq |x| + |y|$.

Next, pull the absolute value inside the sum:

$$|A_\ell(t)| \leq (1 - \phi)^t |A_\ell(0)| + \delta \sum_{\tau=1}^t (1 - \phi)^{t-\tau} |\tilde{r}_\tau| \mathbf{1}\{\ell = j_\tau\}.$$

This uses the fact that $|\sum u_\tau| \leq \sum |u_\tau|$. Also, $|(1 - \phi)^{t-\tau}| = (1 - \phi)^{t-\tau}$ since $(1 - \phi) \in (0, 1)$, and $|\mathbf{1}\{\ell = j_\tau\}| = \mathbf{1}\{\ell = j_\tau\}$ since the indicator is always 0 or 1.

Now, apply the reward bound and indicator bound. Since $|\tilde{r}_\tau| \leq R$ and $\mathbf{1}\{\ell = j_\tau\} \leq 1$, each term can be bounded as follows:

$$|A_\ell(t)| \leq (1-\phi)^t |A_\ell(0)| + \delta \sum_{\tau=1}^t (1-\phi)^{t-\tau} R \mathbf{1}\{\ell = j_\tau\} \leq (1-\phi)^t |A_\ell(0)| + \delta R \sum_{\tau=1}^t (1-\phi)^{t-\tau}.$$

Reindex the geometric sum by setting $k = t - \tau$. As τ goes from 1 to t , k goes from $t-1$ down to 0. Thus

$$\sum_{\tau=1}^t (1-\phi)^{t-\tau} = \sum_{k=0}^{t-1} (1-\phi)^k.$$

Hence

$$|A_\ell(t)| \leq (1-\phi)^t |A_\ell(0)| + \delta R \sum_{k=0}^{t-1} (1-\phi)^k.$$

Finally, upper-bound the finite geometric series by the infinite one:

$$\sum_{k=0}^{t-1} (1-\phi)^k \leq \sum_{k=0}^{\infty} (1-\phi)^k = \frac{1}{\phi}.$$

This holds because $0 < 1 - \phi < 1$ and all terms are nonnegative, so extending the sum can only increase it. Therefore

$$|A_\ell(t)| \leq (1-\phi)^t |A_\ell(0)| + \frac{\delta R}{\phi}.$$

In summary, the general uniform bound, without assuming a particular initialization, is

$$|A_\ell(t)| \leq (1-\phi)^t |A_\ell(0)| + \frac{\delta R}{\phi} \leq |A_\ell(0)| + \frac{\delta R}{\phi},$$

since $(1-\phi)^t \leq 1$. If we initialize $A_\ell(0) = 0$ as in Algorithm 1, the bound simplifies to

$$|A_\ell(t)| \leq \frac{\delta R}{\phi} \quad \text{for all } t.$$

Even if $A_\ell(0) \neq 0$, the extra term $(1-\phi)^t |A_\ell(0)|$ decays to zero, so the bound tightens to $\delta R/\phi$ over time.

Step 3: Steady state (long-run expectation)

We begin again from the one-step update:

$$A_\ell(t) = (1-\phi)A_\ell(t-1) + \delta \tilde{r}_t \mathbf{1}\{\ell = j_t\}.$$

Taking expectations on both sides and applying linearity gives

$$\mathbb{E}[A_\ell(t)] = (1-\phi) \mathbb{E}[A_\ell(t-1)] + \delta \mathbb{E}[\tilde{r}_t \mathbf{1}\{\ell = j_t\}].$$

Now compute $\mathbb{E}[\tilde{r}_t \mathbf{1}\{\ell = j_t\}]$ by conditioning on j_t :

$$\mathbb{E}[\tilde{r}_t \mathbf{1}\{\ell = j_t\}] = \mathbb{E}[\mathbb{E}[\tilde{r}_t \mathbf{1}\{\ell = j_t\} \mid j_t]] = \mathbb{E}[\mathbf{1}\{\ell = j_t\} \mathbb{E}[\tilde{r}_t \mid j_t]].$$

By the definitions under stationarity, let $p_\ell = \Pr(j_t = \ell)$ and $\mu_\ell = \mathbb{E}[\tilde{r}_t \mid j_t = \ell]$. Then

$$\mathbb{E}[\tilde{r}_t \mathbf{1}\{\ell = j_t\}] = \sum_j \Pr(j_t = j) \mathbf{1}\{\ell = j\} \mathbb{E}[\tilde{r}_t \mid j_t = j] = p_\ell \mu_\ell.$$

Substituting back, the recursion for expectations becomes

$$\mathbb{E}[A_\ell(t)] = (1-\phi) \mathbb{E}[A_\ell(t-1)] + \delta p_\ell \mu_\ell.$$

Letting $y_t := \mathbb{E}[A_\ell(t)]$, $\alpha := 1 - \phi \in (0, 1)$, and $c := \delta p_\ell \mu_\ell$, the recursion is

$$y_t = \alpha y_{t-1} + c.$$

The general solution to this linear difference equation is

$$y_t = \alpha^t y_0 + \frac{c}{1 - \alpha} (1 - \alpha^t).$$

Since $\alpha = 1 - \phi \in (0, 1)$, we have $\alpha^t \rightarrow 0$ as $t \rightarrow \infty$. Therefore

$$\lim_{t \rightarrow \infty} \mathbb{E}[A_\ell(t)] = \frac{c}{1 - \alpha} = \frac{\delta p_\ell \mu_\ell}{\phi}.$$

Equivalently, we can check the fixed point directly: setting $y_t = y_{t-1} = y_\star$ in the recursion yields

$$y_\star = \alpha y_\star + c,$$

whose solution is $y_\star = \frac{c}{1 - \alpha} = \frac{\delta p_\ell \mu_\ell}{\phi}$. Since $|\alpha| < 1$, the recursion is a contraction and converges to this unique fixed point. □

A.2 Proof of Lemma 4.2

Lemma 4.2[Controlled attention drift under a bounded bias]

Let $p = \text{softmax}(z) \in \Delta^n$ and $q = \text{softmax}(z + b)$ for any bias b with $\|b\|_\infty \leq \varepsilon$. Then the total variation distance satisfies

$$\|q - p\|_{\text{TV}} \leq \tanh(\varepsilon).$$

Proof. We begin by recalling definitions. The softmax distribution is given by $p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$ and $q_i = \frac{e^{z_i + b_i}}{\sum_j e^{z_j + b_j}}$. The total variation (TV) distance is $\|q - p\|_{\text{TV}} = \frac{1}{2} \sum_i |q_i - p_i|$. This measures the discrepancy between two probability distributions, taking values in $[0, 1]$.

To analyze this bound, we first express q in terms of p . Multiplying and dividing by $\sum_k e^{z_k}$, one obtains

$$q_i = \frac{e^{z_i + b_i}}{\sum_j e^{z_j + b_j}} = \frac{e^{b_i}}{\sum_j p_j e^{b_j}} p_i.$$

Letting $Z = \sum_j p_j e^{b_j}$ and $\lambda_i = e^{b_i}/Z$, this simplifies to $q_i = \lambda_i p_i$. Furthermore, $\sum_i p_i \lambda_i = 1$, so the p -weighted average of the reweighting factors λ_i is exactly one.

Next, we bound the reweighting factors. Since $b_i \in [-\varepsilon, \varepsilon]$, it follows that $e^{b_i} \in [e^{-\varepsilon}, e^\varepsilon]$. The denominator Z is a convex combination of such terms, so $Z \in [e^{-\varepsilon}, e^\varepsilon]$. Therefore

$$\lambda_i = \frac{e^{b_i}}{Z} \in \left[\frac{e^{-\varepsilon}}{e^\varepsilon}, \frac{e^\varepsilon}{e^{-\varepsilon}} \right] = [e^{-2\varepsilon}, e^{2\varepsilon}],$$

so each likelihood ratio q_i/p_i is bounded between $a = e^{-2\varepsilon}$ and $b = e^{2\varepsilon}$ with $ab = 1$.

Now we express the TV distance in terms of these reweighting factors:

$$\|q - p\|_{\text{TV}} = \frac{1}{2} \sum_i |q_i - p_i| = \frac{1}{2} \sum_i p_i |\lambda_i - 1|.$$

Partitioning indices into $S = \{i : \lambda_i \geq 1\}$ and S^c , and using the fact that $\sum_i p_i \lambda_i = 1$ implies $\sum_i p_i (\lambda_i - 1) = 0$, we deduce that

$$\|q - p\|_{\text{TV}} = \sum_{i \in S} p_i (\lambda_i - 1) = \sum_{i \in S^c} p_i (1 - \lambda_i).$$

Thus TV reduces to the total “mass shifted upwards” (or downwards), and both are equal.

To maximize this deviation under the constraints $\lambda_i \in [a, b]$ and $\sum_i p_i \lambda_i = 1$, one only needs to consider extreme points where each λ_i equals either a or b . The worst case occurs with a two-point mixture: set $\lambda = b$ on some fraction s of the p -mass and $\lambda = a$ on the rest. Enforcing $\sum_i p_i \lambda_i = 1$ yields $1 = sb + (1 - s)a$, hence $s = \frac{1-a}{b-a}$. Substituting into the expression for TV gives

$$\|q - p\|_{\text{TV}} = \frac{1}{2}(s(b-1) + (1-s)(1-a)) = \frac{(1-a)(b-1)}{b-a}.$$

Finally, substituting $a = e^{-2\varepsilon}$ and $b = e^{2\varepsilon}$, we compute

$$\|q - p\|_{\text{TV}} = \frac{b-1}{b+1} = \frac{e^{2\varepsilon} - 1}{e^{2\varepsilon} + 1} = \tanh(\varepsilon).$$

This establishes the desired bound. Intuitively, the perturbation b shifts logits by at most $\pm\varepsilon$, so each attention probability can change by a factor in $[e^{-2\varepsilon}, e^{2\varepsilon}]$. The worst-case change occurs when some entries receive the full amplification while others receive the full suppression, balanced to maintain normalization. This yields the clean $\tanh(\varepsilon)$ formula. For small ε , the bound behaves linearly, $\tanh(\varepsilon) \approx \varepsilon$, so clipping the bias ensures the drift remains small and interpretable. $\square$

B Hyperparameters for EWA-VQ-ODT

Unless noted, all runs use the same backbone and optimizer as the ODT baseline. The values below reflect the configurations used in our main experiments (scripts `run_quick1_ewa.py` and `run_quick2_ewa.py`).

B.1 Core EWA-VQ Settings

Parameter	Value	Description
Attraction decay ϕ	0.05	Per-step global decay: $A \leftarrow (1 - \phi)A$
Chosen weight δ	0.8	Additive update for routed code: $A_{j_t} += \delta \tilde{r}_t$
Attention bias scale β	0.05	Scales the attraction bias added to action columns
Trajectory length (max)	1000	Max steps processed per trajectory
VQ codebook size M	27	Number of discrete action surrogates (codes)
Grid bins factor	1.0	Adaptive bins per action dim (see below)
Routing	direct lookup	Grid cell $\rightarrow$ code index via cached table T

Codebook/grid construction. We use an axis-aligned grid with B bins per dimension; for the tasks studied we set $B=3$. The offline table $T : \{0, \dots, B^d - 1\} \rightarrow \{1, \dots, M\}$ maps each grid cell to its nearest code. The table is cached per (d, B, M, env) and reused.

B.2 Environment-Specific Settings

Env	Action dim	B	Cells B^d	M used	Online RTG	Eval RTG
<code>hopper-medium-v2</code>	3	3	27	27 (full coverage)	7200	3600
<code>walker2d-medium-replay-v2</code>	6	3	729	27 (subset)	10000	5000

Env	Eval context length	Ordering (pos. emb.)	Notes
<code>hopper-medium-v2</code>	20	1	Positional embeddings enabled
<code>walker2d-medium-replay-v2</code>	5	0	No positional embeddings

Optional configuration (not in main results). For 6D tasks like `halfcheetah-medium-expert-v2`, we use the same $B=3$ (729 cells), $M=27$ routed codes, and (online/eval) RTGs of (12000, 6000).

Training	Value	Details
Online iterations (quick runs)	10	Evaluate every 2 iterations
Updates per iteration	30	Per online iteration
Batch size	64	Mini-batch size
Context length K (train)	20	DT/ODT context
Eval episodes / checkpoint	10	Parallel eval environments
Seeds	3	Reported as averages

Backbone/Opt	Value	Details
Embedding dim	512	Token embedding size
Transformer layers	4	Encoder blocks
Attention heads	4	Multi-head self-attention
Activation	ReLU	
Dropout	0.1	Residual/MLP dropout
Initial temperature	0.1	As in code (for logits scaling)
Optimizer	Adam	
Learning rate	1×10^{-4}	With warmup
Weight decay	5×10^{-4}	
Warmup steps	10,000	Linear warmup schedule
Device	CUDA	GPU training

B.3 Training and Model Hyperparameters

Grid adaptation rules. For $d \leq 6$, we use $B=3$ bins/dim. (General logic: $B \in [2, 8]$ with a cap on M —here $M=27$; max M can be raised up to 128 if memory allows.) The offline T table is built once (GPU-batched L_2) and cached.

Implementation note. Attractions $A \in \mathbb{R}^M$ are stored on GPU; each step applies one vectorized decay and a single indexed add for the routed code. The attention bias adds βA only to action-token columns prior to masking and softmax. ::contentReference[oaicite:0]index=0